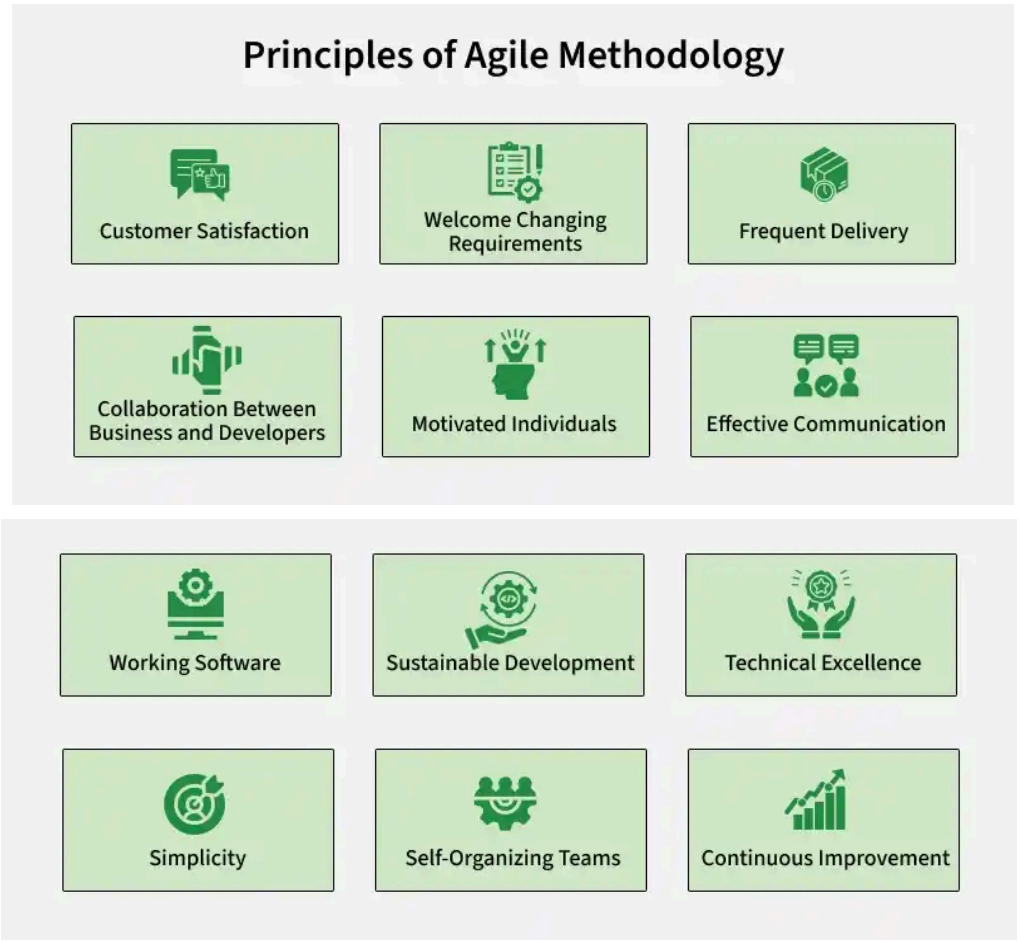
What is the Agile methodology?

The Agile methodology is an approach that divides work into phases, emphasizing continuous delivery and improvement. Agile benefits teams by enabling adaptive planning, rapid execution, and ongoing evaluation, leading to more responsive and successful outcomes.

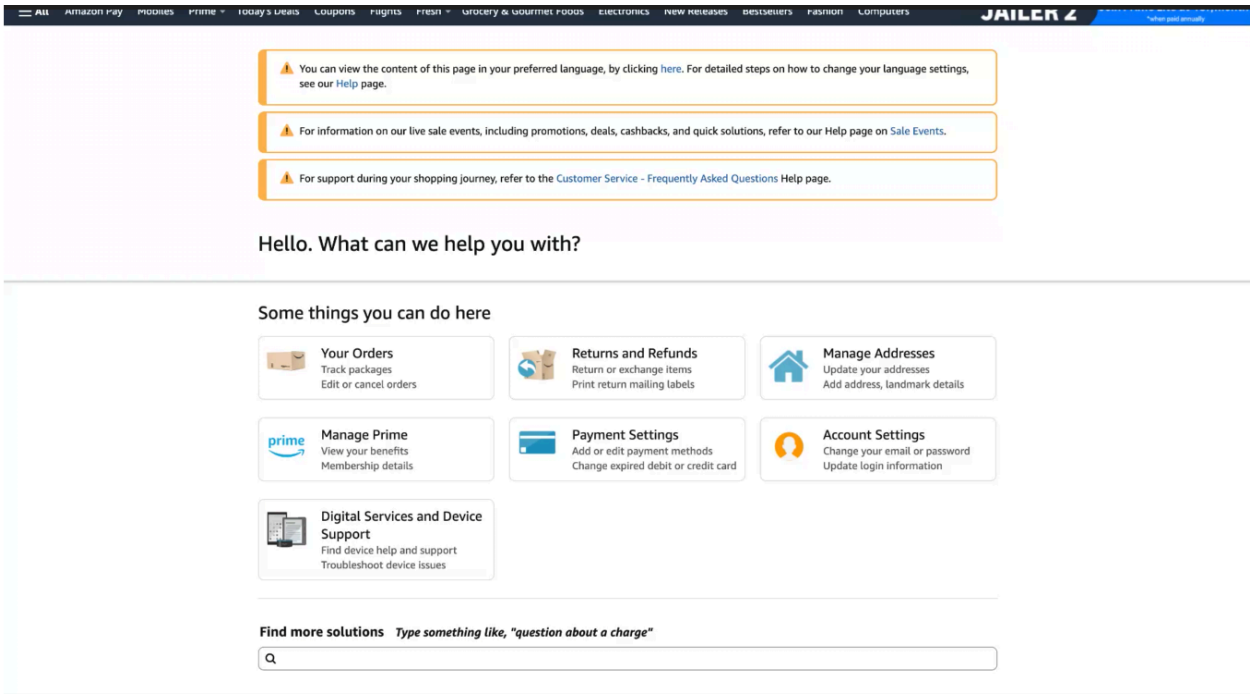
Agile is an approach to software development that emphasizes iterative development, incremental delivery, customer collaboration, continuous feedback, and responding to changing requirements.

- Supports frequent and incremental delivery of valuable software.
- Encourages collaboration among customers, stakeholders, and development teams.
- Supports adapting development work based on feedback and changing requirements.



Explaining agile methodology based on amazon customer service

Here's a deeper walkthrough, using your Amazon Customer Service example at every step so the concepts stay concrete.



1. Why Agile exists: the problem it solves

Before Agile, most software was built using the Waterfall model. Work flowed in one direction through fixed phases: gather all requirements, design the entire system, build everything, test everything, then release. For your Customer Service portal, Waterfall would mean writing a complete specification for Orders, Returns, Addresses, Prime, Payments, Account Settings, and Device Support, spending months building all of it, and only then showing it to customers.

That approach has serious weaknesses. Requirements change while you're building (Amazon might launch a new return policy mid-project). Mistakes in early phases are discovered late, when they're expensive to fix. And customers get nothing useful until the very end, which might be a year away.

Agile flips this. You build a small, working piece, release it, learn from real usage, and use that learning to decide what to build next. Instead of one big delivery, you have many

small ones. The 2001 Agile Manifesto captured this in four value statements: individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. The items on the right still matter; the items on the left matter more.

Agile is a philosophy, and Scrum is the most popular framework for practising it. Most of what follows describes Scrum, since your example (epics, stories, tasks, hour estimates) fits it naturally.

2. Breaking down the work: the hierarchy in detail

Epic: "Customer Service"

An epic is a large body of work tied to a business goal. Here the goal is something like: *reduce customer support calls by letting customers solve common problems themselves.* An epic is too big to plan precisely or finish in one sprint. It might span several months and many teams. Its main job is to group related work and keep everyone aligned on the bigger purpose.

Features: the tiles on the page

Each tile in your screenshot is a feature: Your Orders, Returns and Refunds, Manage Addresses, Manage Prime, Payment Settings, Account Settings, Digital Services. A feature is a coherent capability that delivers value on its own, which is what makes incremental release possible. Amazon could launch the help page with only "Your Orders" working and the other tiles marked "coming soon," and customers would still benefit.

Features also help with prioritisation. If data shows that 40% of support calls are "where is my package?", then Your Orders clearly comes first, while Manage Prime can wait.

User stories: value from the customer's perspective

A user story is the smallest piece of work that delivers something meaningful to a user, and it should fit within a single sprint. Stories are deliberately written from the user's point of view rather than the developer's, using this template:

> As a [type of user], I want [some goal], so that [some benefit].

Applying that to your stories:

Story title	Full user story
Track Packages	As a customer, I want to see where my package is, so that I know when to expect delivery.
Edit or Cancel Orders	As a customer, I want to change or cancel an order before it ships, so that I don't receive something I no longer need.
Return and Exchange Items	As a customer, I want to return or exchange an item I'm unhappy with, so that I get a refund or the right product.
Print Labels	As a customer, I want to print a return shipping label, so that I can send the item back without visiting a store.

The "so that" clause is not a decoration. It tells the team *why* the work matters, which guides design decisions. If the goal of tracking is "know when to expect delivery," then the expected delivery date should be the most prominent thing on screen, not the warehouse name.

Acceptance criteria define exactly when a story is complete. For Track Packages they might be: the customer can see all orders from the last 90 days; each order shows its current status (Ordered, Shipped, Out for Delivery, Delivered); shipped orders show the carrier and a tracking number; the expected delivery date is displayed; and if the package is delayed, a message explains why. These criteria give developers a clear target and give testers something objective to verify.

Good stories follow the INVEST checklist. They should be Independent (deliverable without waiting on another story), Negotiable (details can be discussed), Valuable (useful to the user), Estimable (the team can size it), Small (fits in a sprint), and Testable (you can confirm it works). "Edit or Cancel Orders" might fail the Small test once examined, in which case the team would split it into two stories, "Edit Order" and "Cancel Order." Notice your task list already treats it as just "Edit Orders," which suggests that split has effectively happened.

Tasks: the technical work

Tasks are what developers actually do to complete a story. Your five tasks represent the full stack:

- ➢ Build the React UI (5 hrs) — the screen customers see: an order list, status badges, a timeline of tracking events.

- ➢ Build the FastAPI layer (6 hrs) — the backend endpoints, such as `GET /orders` and `GET /orders/{id}/tracking`, that the React app calls to fetch data.
- ➢ Build the DB schema (8 hrs) — tables for orders, shipments, and tracking events, plus their relationships.
- ➢ Write unit tests (4 hrs) — automated tests confirming the API returns correct data and the UI handles cases like "no orders found."
- ➢ Configure the CI/CD pipeline (2 hrs) — automation that runs the tests and deploys the feature whenever code is merged.

This is called a vertical slice: one story cuts through every layer of the system. The alternative, a horizontal approach, would be "build all database tables for the whole portal this sprint, all APIs next sprint, all UI after that." That delays any working software for months, which defeats the purpose of Agile. With vertical slicing, at the end of the first sprint a customer can genuinely track a package.

3. The people: Scrum roles

The Product Owner owns the "what" and "why." They represent the business and customers, maintain the backlog, and decide priority. In your example, they'd be the person deciding that Track Packages comes before Manage Prime, and the one who writes or approves the acceptance criteria.

The Scrum Master owns the process. They're not a traditional manager; they facilitate meetings, remove blockers (for example, chasing the carrier-integration team when the tracking API access is delayed), and coach the team on Agile practices.

The Development Team owns the "how." It's typically 3–9 people with all the skills needed to deliver a story end to end: here, a React developer, a Python/FastAPI developer, someone comfortable with database design, and someone who handles testing and DevOps. In a mature team people overlap; the backend developer might help write tests if testing is the bottleneck. The team decides how much work it can take on and how to do it.

4. The artifacts: what the team keeps track of

The Product Backlog is a single, ordered list of everything that might be built: all features and stories across the whole epic. The top is detailed and ready to work on; the bottom is vague and will be refined later. For your portal it might look like this:

Priority	Story	Status
1	Track Packages	Refined, estimated
2	Edit Orders	Refined, estimated
3	Cancel Orders	Refined, estimated
4	Return and Exchange Items	Needs refinement
5	Print Labels	Needs refinement
6	Manage Addresses	Rough idea
7	Manage Prime	Rough idea

The Sprint Backlog is the subset of stories the team commits to for the current sprint, broken down into tasks.

The Increment is the working, tested software produced by the end of the sprint. It must meet the team's Definition of Done, a shared checklist that applies to every story. For example: code reviewed by a peer, unit tests written and passing, acceptance criteria verified, deployed to staging via CI/CD, and no known critical bugs. The difference between acceptance criteria and Definition of Done is worth stating clearly: acceptance criteria are specific to one story, while the Definition of Done is the quality bar for every story.

5. The rhythm: Scrum events in a sprint

A sprint is a fixed time box, usually two weeks. Here's how one would play out for your team.

Backlog refinement (ongoing, before the sprint)

The team and Product Owner review upcoming stories, clarify questions, split large ones, and estimate them. This is where someone asks, "For Track Packages, do we show orders from all time or just recent ones?" and the Product Owner answers "last 90 days," which becomes an acceptance criterion.

Sprint planning (day 1, a few hours)

The team answers two questions: *what* can we deliver this sprint, and *how* will we do it? The Product Owner proposes a sprint goal, perhaps "Customers can track and edit their orders without contacting support." The team then checks capacity.

Suppose the team has four developers. Over a two-week sprint, each has roughly 60 working hours, but after meetings, code reviews, and unexpected issues, maybe 30–35 hours per person go into planned tasks. That's around 130 hours of capacity. Your two stories total 50 hours, leaving room for a third, such as Cancel Orders, or a buffer for risks like learning a new carrier API.

Daily standup (every day, 15 minutes)

Each person briefly shares what they finished yesterday, what they'll work on today, and anything blocking them. A typical day-3 standup might sound like: "I finished the orders and shipments tables yesterday; today I'll add the tracking events table. No blockers." Or: "I'm building the tracking endpoint but I don't have credentials for the carrier's sandbox. That's blocking me." The Scrum Master takes that blocker and chases it after the meeting. The standup is for coordination, not detailed problem-solving.

The sprint board

Progress is usually visible on a board, physical or in a tool :

To Do	**In Progress**	**In Review**	**Done**
Edit Orders: React UI	Track Packages: FastAPI layer	Track Packages: DB schema	
Edit Orders: FastAPI layer	Track Packages: Unit tests		
Edit Orders: Unit tests			

Tasks move left to right. Anyone, including stakeholders, can glance at the board and see where things stand.

Sprint review (last day)

The team demonstrates working software to stakeholders: a live order list showing a package as "Out for Delivery" with its expected date. Stakeholders give feedback, perhaps "customers will want SMS notifications when status changes." That doesn't get squeezed into the current sprint; it becomes a new story in the backlog for the Product Owner to prioritise.

Sprint retrospective (last day)

The team reflects on *how* they worked, not what they built. What went well? ("Vertical slicing meant we had something doable by day 6.") What didn't? ("The DB schema took 12 hours instead of 8 because requirements weren't clear.") What will we change? ("Next sprint, we'll draft the schema together during refinement.") This continuous self-improvement is one of the most valuable parts of Agile.

Then the next sprint begins, perhaps tackling Returns and Refunds.

6. Estimation and measuring progress

Hours versus story points

This example uses hours, which works well at the task level. At the story level, many teams use story points, a relative measure of effort, complexity, and uncertainty, often on a Fibonacci-like scale (1, 2, 3, 5, 8, 13). The reasoning is that people are poor at predicting absolute time but good at comparing: "Print Labels feels about half as complex as Track Packages." If Track Packages is an 8, Print Labels might be a 3.

Teams often estimate using planning poker: each member privately picks a card, everyone reveals at once, and large differences trigger discussion. If the frontend developer says 3 and the backend developer says 13 for Return and Exchange Items, the conversation might reveal that refunds require integrating with the payment system, which the frontend developer hadn't considered. That discussion is often more valuable than the number itself.

Velocity

Velocity is the number of story points a team completes per sprint, averaged over several sprints. If your team consistently finishes about 30 points per sprint and the remaining backlog for the epic totals 150 points, you can forecast roughly five more sprints. This forecast improves with every sprint, which is far more reliable than a guess made at the start of a year-long Waterfall project.

Burndown chart

A burndown chart plots the work remaining in the sprint against time. Ideally it slopes steadily down to zero by the last day. If the line stays flat for several days, something is blocked, and the team knows to act early rather than discovering the problem at the deadline.

7. What this example teaches about Agile in practice

Your Track Packages and Edit Orders stories have identical task lists and estimates, and that's a useful learning moment. Once the database schema for orders exists from Track Packages, Edit Orders would only need to extend it, perhaps 2 hours instead of 8. Likewise, the CI/CD pipeline is typically configured once for the project and only adjusted for each new feature. In Agile, the team would notice this during refinement or planning and re-estimate, and over time their estimates get more accurate. Adapting the plan as you learn is the whole point.

It's also worth noticing what's missing. There are no tasks for code review, integration with the carrier's tracking service, error handling, or accessibility. A good Definition of Done would surface these, and the retrospective is where the team would add them to future planning.

8. Putting it all together

The full cycle for your Customer Service portal looks like this: the Product Owner holds the vision (the epic) and orders the backlog by value; features break the vision into deliverable capabilities; user stories express what each capability means to a customer, with acceptance criteria defining success; tasks describe the technical work in vertical slices; the team plans a sprint, works in short daily cycles, demonstrates working software, reflects, and improves. Each sprint ends with something real a customer could use, and each sprint's feedback shapes the next.

So instead of launching the entire help page you see in the screenshot after a year, Amazon could ship order tracking in two weeks, editing and cancelling in four, returns in six, and keep refining based on how customers actually use it.